Minty NS: Comprehensive Project Report

Project Documentation

April 17, 2026

Abstract

Minty NS is an interactive network simulation and graph-analysis platform that combines a C++ simulation core, a Node.js backend bridge, and a React frontend visualization layer. This report presents the full project flow, implementation languages, major components, measured output results, and step-by-step instructions to run and use the project.

1 Project Overview

Minty NS enables users to:

- build or generate network topologies,
- run packet-flow simulation with configurable traffic and timing parameters,
- inspect runtime event logs and queue behavior,
- compute and compare graph algorithms (Shortest Path and MST), and
- analyze quality metrics such as latency, throughput, packet loss, jitter, and hop count.

2 Languages and Technology Stack

2.1 Language by Layer

Layer	Language	Purpose
Frontend	JavaScript (React)	User interface, graph visualization, controls, and API calls
Backend API	JavaScript (Node.js/Express)	REST endpoints, process orchestration, JSON response handling
Core Simulation	C++17	High-performance packet simulation and event processing
Graph Analysis	C++17	Dijkstra, Bellman-Ford, BFS, Kruskal, Prim, and complexity timing
Build System	Make + npm scripts	Native compilation and JS package scripts

Table 1: Main implementation languages and responsibilities

2.2 Important Frameworks and Libraries

- Frontend: React, Vite, React Flow, Axios, D3 Force.
- Backend: Express, body-parser, cors, child_process spawn.
- Native: C++ standard library and project-specific algorithm/simulation modules.

3 Project Architecture and Flow

3.1 High-Level Architecture

1. **Frontend** sends simulation or analysis requests.
2. **Backend** receives requests, writes input files, and launches native C++ binaries.
3. **C++ engine** executes simulation/analysis and writes outputs.
4. **Backend** packages outputs into JSON and returns to frontend.
5. **Frontend** renders topology, logs, and metrics.

3.2 Detailed Runtime Flow

A. Simulation Flow (/run)

1. User configures topology, source/destination, traffic, and packet parameters in the UI.
2. Frontend submits POST /run with `configOverrides` and optional custom topology text.
3. Backend writes input files to `data_input/`.
4. Backend launches `build/simulator` using child process spawn.
5. Simulator executes event-driven packet flow and writes outputs such as:
   - `data_output/result.json`
   - `simulation_console.log`
   - `simulation_events.csv`
   - `simulation_events.log`
6. Backend reads artifacts, merges data, and returns JSON payload to frontend.
7. Frontend visualizes event progression, queue state, and simulation metrics.

B. Analysis Flow (/analyze)

1. Frontend submits `nodeCount`, `source`, `dest`, and weighted edges.
2. Backend writes edge data to `data_input/topology_edges.json`.
3. Backend launches `build/analysis-tool`.
4. Analysis tool computes:
   - shortest path and cost,
   - MST (Kruskal and Prim),
   - complexity timing summaries,
   - BFS distance and Bellman-Ford status.
5. Backend returns structured analysis JSON to frontend panels.

4 Measured Results Snapshot

This section uses the latest local output snapshot from project artifacts.

4.1 Simulation Metrics Snapshot

Source: `data_output/result.json`

Metric	Value
Average latency	6.31667 ms
Throughput	16384 bit/ms
Packet loss rate	0.205298
Packets sent	151
Packets delivered	120
Packets dropped	31
Average hops	3
Jitter	0.465176 ms
Minimum latency	6 ms
Maximum latency	7 ms
Event count in log	2788

Table 2: Recent simulation output summary

4.2 Algorithm Analysis Snapshot

Source: `build/analysis-tool 10 0 9 data_output/topology_edges.json`

Result	Value
Shortest path (Dijkstra)	$0 \to 2 \to 3 \to 9$
Shortest path cost	21
Shortest path hops	3
MST total cost (Kruskal)	65
MST total cost (Prim)	65
Path stretch (MST vs shortest)	1.0
Bellman-Ford negative-cycle check	true
BFS distance	3

Table 3: Recent graph-analysis output summary

4.3 Interpretation

- The shortest path and MST path were identical for the selected endpoints in the sampled topology (stretch = 1.0).
- Packet delivery was successful for most packets, with measurable but controlled drop/loss behavior.
- Millisecond-level algorithm timing indicates low overhead for small-to-medium topology snapshots.

5 How to Build and Run the Project

5.1 Prerequisites

- Node.js 18+ and npm
- g++ with C++17 support
- make

5.2 Step-by-Step Setup

```
# 1) Install root dependencies
npm install

# 2) Install frontend dependencies
npm --prefix frontend install

# 3) Build native binaries
make release
```

5.3 Start Development Mode

```
# Starts backend (port 3000) + frontend (Vite, usually 5173)
npm run dev
```

5.4 Run Backend or Frontend Separately

```
# Backend only
npm run dev:backend

# Frontend only
npm run dev:frontend
```

5.5 Run Native Simulator Directly

```
./build/simulator config/config.txt
```

5.6 Code for 10k+ Nodes

This section provides practical code snippets for large-scale runs.

A. 10k+ Node Config (Tree Topology)

Create a large-run config (sparse and stable for baseline testing):

```
cat > config/config_10k.txt <<'EOF'
nodes=10000
topology=tree
probability=0.0008
traffic_type=constant
rate=50
packet_size=512
```

```
simulation_time=60
bandwidth=100
delay=2
real_time_mode=0
time_scale=1.0
max_sleep_ms=50
enable_retransmission=1
rtx_timeout_ms=50
max_retransmissions=3
enable_profiling=1
enable_parallel_routing=0
max_nodes=20000
source=0
destination=9999
EOF
```

Run it:

```
./build/simulator config/config_10k.txt
```

B. 10k+ Node Run Through Backend API

```
curl -X POST http://localhost:3000/run \
  -H "Content-Type: application/json" \
  -d '{
    "configOverrides": {
      "nodes": 10000,
      "max_nodes": 20000,
      "topology": "tree",
      "traffic_type": "constant",
      "rate": 50,
      "packet_size": 512,
      "simulation_time": 60,
      "bandwidth": 100,
      "delay": 2,
      "source": 0,
      "destination": 9999
    }
  }'
```

C. Optional Sparse Custom Topology for 10k+ Nodes

Generate a sparse topology file (chain + skip links) to keep memory usage controlled:

```
node - <<'NODE'
const fs = require('fs');
const N = 10000;
const lines = [];

// Connectivity backbone
for (let i = 0; i < N - 1; i++) lines.push(`${i} ${i + 1} 1`);

// Extra sparse shortcuts
for (let i = 0; i + 50 < N; i += 10) lines.push(`${i} ${i + 50} 2`);

fs.writeFileSync('data_input/topology_10k.txt', lines.join('\n'));
console.log(`Wrote ${lines.length} edges`);
```

```
NODE
```

Run simulator with custom topology:

```
./build/simulator config/config_10k.txt data_input/topology_10k.txt
```

D. 10k+ Node Analysis Tool Run

```
./build/analysis-tool 10000 0 9999 data_output/topology_edges.json \
  > data_output/analysis_10k.json
```

Scaling note: for random topologies at 10k+, keep edge probability very small (for example $p \leq 0.001$) to avoid excessive edge counts and long runtimes.

E. 10k+ Measured Results (Current Run)

Source commands:

- `./build/simulator config/config_10k.txt`
- `./build/analysis-tool 10000 0 9999 data_output/topology_edges.json`
- Output saved to `data_output/analysis_10k.json`

10k Simulation Metric	**Value**
Average latency	18.4678 ms
Throughput	142131 bit/ms
Packet loss rate	0.306
Packets sent	3000
Packets delivered	2082
Packets dropped	918
Average hops	9
Jitter	0.498963 ms
Minimum latency	18 ms
Maximum latency	19 ms
Event count	112836

Table 4: 10k simulation results from `config/config_10k.txt`

Note: timing values can vary by CPU, compiler flags, and background system load.

5.7 Typical User Workflow in the UI

1. Open the frontend in browser.
2. Load or edit configuration.
3. Set source and destination nodes.
4. Build/run simulation.
5. Inspect queue/events/metrics panels.
6. Open Path and MST tabs for algorithm comparisons.

10k Analysis Result	Value
Graph size	$V = 10000, E = 9999$
Shortest path cost	54
Shortest path hops	9
MST total cost (Kruskal)	54813
MST total cost (Prim)	54813
Path stretch (MST vs shortest)	1.0
Bellman-Ford negative-cycle check	true
BFS distance	9
Dijkstra measured time	0.67665 ms
Bellman-Ford measured time	291.796 ms
BFS measured time	0.079398 ms
Kruskal measured time	0.335134 ms
Prim measured time	0.773891 ms

Table 5: 10k graph-analysis results from `data_output/analysis_10k.json`

6 API Usage Examples

Get Current Config

```
curl http://localhost:3000/config
```

Run Simulation

```
curl -X POST http://localhost:3000/run \
  -H "Content-Type: application/json" \
  -d '{
    "configOverrides": {
      "nodes": 200,
      "topology": "tree",
      "traffic_type": "constant",
      "rate": 5,
      "packet_size": 512,
      "simulation_time": 30,
      "source": 0,
      "destination": 199
    }
  }'
```

Run Algorithm Analysis

```
curl -X POST http://localhost:3000/analyze \
  -H "Content-Type: application/json" \
  -d '{
    "nodeCount": 5,
    "source": 0,
    "dest": 4,
    "edges": [
      {"from":0,"to":1,"weight":2},
      {"from":1,"to":4,"weight":5},
      {"from":0,"to":2,"weight":1}
```

```
    ]
  }'
```

7 Build Targets and Artifacts

7.1 Useful Build Targets

```
make
make debug
make release
make clean
make rebuild
make help
```

7.2 Key Generated Artifacts

- `build/simulator`
- `build/backend-runner`
- `build/analysis-tool`
- `data_output/result.json`
- `simulation_console.log`
- `simulation_events.csv`

8 How to Compile This .tex Report

```
# Example with pdflatex
pdflatex Minty_NS_report.tex
pdflatex Minty_NS_report.tex
```

9 Conclusion

Minty NS demonstrates a practical full-stack systems workflow where high-performance C++ simulation and algorithms are exposed through a Node.js API and consumed by an interactive React UI. The architecture supports both educational analysis and scalable experiment workflows, with measurable runtime results and clear operational flow from configuration to visualization.